

OS-Lab4

思考题

Thinking4.1

- 1.系统调用流程 `sys_*->msyscall->exc_gen_entry` 中的 `SAVE_ALL` 保护了所有的通用寄存器。
2. `$a0 - $a3` 没有被重新赋值，可以直接调用 `msyscall` 留下信息
- 3.通过内核的系统调用分发函数完成。该函数会从保存好的 `Trapframe` 结构中提取出用户最初存入的参数，然后把这些取出的参数作为实参，直接传递给对应的 `sys_*` 函数。
4. `EPC += 4` :跳过 `syscall` 执行下一条指令；把返回值赋给 `$v0` :使用户能直接从 `$v0`

Thinking4.2

根据如下代码：

```
u_int mkenvid(struct Env *e) {  
    static u_int i = 0;  
    return ((++i) << (1 + LOG2NENV)) | (e - envs);  
}
```

以及如下宏：

```
#define LOG2NENV 10  
#define NENV (1 << LOG2NENV)  
#define ENVX(envid) ((envid) & (NENV - 1))
```

`ENVX` 是取 `envid` 的后十位，`envid` 和 `envs` 并不一一对应，需要进行反向确认。

Thinking4.3

`envid2env` :

```
int envid2env(u_int envid, struct Env **penv, int checkperm) {
    struct Env *e;
    if (envid == 0) {
        *penv = curenv;
        return 0;
    }
    e = &envs[ENVX(envid)];
}
```

envid=0 代表 curenv。若可以返回0，则 envid2env(0,&env,0) 只能取到 curenv 而不能取到 envid 确实为0的情况，影响进程间通信的正常实现。

Thinking4.4

C

fork 之前只有父进程存在。fork 之后，父子进程同时开始执行之后代码。

fork 代码本身是父进程执行的。在父子进程的返回值分别是子进程pid与0。

Thinking4.5

env_init:

```
base_pgdir = (Pde *)page2kva(p);
map_segment(base_pgdir, 0, PADDR(pages), UPAGES, ROUND(npag * sizeof(struct Page), PAGE_SIZE),
map_segment(base_pgdir, 0, PADDR(envs), UENVS, ROUND(NENV * sizeof(struct Env), PAGE_SIZE), PTE_
```

UXSTACK 是用户态异常处理的专用站，如果共享，同时发生异常时会相互覆盖异常上下文和局部变量。如果 UXSTACK 本身也是COW的，写入操作会再次触发缺页异常，导致死循环。

UVPT/VPT/VPD 是利用自映射机制将进程自身的物理页表映射到虚拟内存特殊区域，不在 duppage 拷贝的普通内存范围内。如果务处理，会导致紫禁城内存管理操作完全错乱。

Thinking4.6

lib.h:

```
#define vpt ((const volatile Pte *)UVPT)
#define vpd ((const volatile Pde *) (UVPT + (PDX(UVPT) << PGSHIFT)))
```

vpt 是用户页表起始位置

vpd 利用自映射，指向页目录在二级页表中位置

存取自身页表方法: vpd[PDX(va)] & vpt[vpn]

page_insert 中 *pte = page2pa(pp) | perm | PTE_C_CACHEABLE | PTE_V ,不具有修改页表项的权限。

Thinking4.7

do_tlb_mod :

```
void do_tlb_mod(struct Trapframe *tf) {
    struct Trapframe tmp_tf = *tf;
    if (tf->regs[29] < USTACKTOP || tf->regs[29] >= UXSTACKTOP) {
        tf->regs[29] = UXSTACKTOP;
    }
    tf->regs[29] -= sizeof(struct Trapframe);
    *(struct Trapframe *)tf->regs[29] = tmp_tf;
    Pte *pte;
    page_lookup(cur_pgdir, tf->cp0_badvaddr, &pte);
    if (curenv->env_user_tlb_mod_entry) {
        tf->regs[4] = tf->regs[29];
        tf->regs[29] -= sizeof(tf->regs[4]);
        tf->cp0_epc = curenv->env_user_tlb_mod_entry;
    } else {
        panic("TLB Mod but no user handler registered");
    }
}
```

COW处理函数自身某个环节触发的异常时就会触发异常重入。

用于由用户态实现现场的恢复。

Thinking4.8

优势：

- 用户可以相对自由的控制异常的产生和处理。
- 减少状态切换，提高运转效率。
- 与内核独立，保证内核安全。

Thinking4.9

写时复制需要先把共享页设为只读，再靠异常处理函数来捕获写操作、完成页复制。如果先设只读再注册处理函数，父进程在设置只读到注册完成的时间窗口内，一旦写入共享页就会触发异常，但此时没有处理函数，会直接导致内核崩溃。

会出现竞态条件：父子进程对只读共享也的写操作触发TLB Mod异常，但没有处理入口，导致的内核崩溃。

难点分析

系统调用

- 发起（用户态）：用户调用 `syscall_xxx`，最终进入 `msyscall` 汇编函数，将系统调用号和参数放入寄存器，执行 `syscall` 指令。
- 陷入（切换）：硬件自动设置 EXL 位（进入内核态），根据异常向量表跳转到 `exc_gen_entry`，调用 `SAVE_ALL` 将当前所有寄存器保存到 `Trapframe`。
- 分发（内核态）：内核函数 `do_syscall` 从 `Trapframe` 中读取系统调用号，在 `syscall_table` 中查找对应的内核处理函数（`sys_xxx`）并执行。
- 返回（恢复）：内核将 `sys_xxx` 的返回值存入子进程 `Trapframe` 的 `$v0` 寄存器，调用 `RESTORE_ALL` 恢复现场，执行 `eret` 指令弹回用户态。

充分理解 `Trapframe` 结构是理解系统调用的关键之一。可以结合 CO 知识理解。

IPC通信

比较简单。知道进程控制块中几个新的数据的含义就很易懂了。

fork

`fork` 是一种用户态下的由用户主动产生新进程的函数。通过系统调用避免内核态与用户态的频繁切换，可以安全地实现内核操作。

`fork`：

```

int fork(void) {
    u_int child;
    u_int i;
    if (env->env_user_tlb_mod_entry != (u_int)cow_entry) {
        try(syscall_set_tlb_mod_entry(0, cow_entry));
    }
    child = syscall_exofork();
    if (child == 0) {
        env = envs + ENVX(syscall_getenvid());
        return 0;
    } for (i = 0; i < PDX(UXSTACKTOP); i++) {
        if (vpd[i] & PTE_V) {
            for (u_int j = 0; j < PAGE_SIZE / sizeof(Pte); j++) {
                u_long va = (i * (PAGE_SIZE / sizeof(Pte)) + j) << PGSHIFT;
                if (va >= USTACKTOP) {
                    break;
                } if (vpt[VPN(va)] & PTE_V) {
                    duppage(child, VPN(va));
                }
            }
        }
    }
    syscall_set_tlb_mod_entry(child, cow_entry);
    syscall_set_env_status(child, ENV_RUNNABLE);
    return child;
}

```

其中比较难理解的是COW（写时复制）机制：

- 核心思想：fork 时只复制页表而不复制物理内存。父子进程共享同一块物理内存，直到其中一方尝试“写”数据。
- 将父子进程的普通可写页全部设为只读，并打上软件标记 PTE_COW。
- 当进程尝试写入时，硬件因无写权限触发 TLB Mod 异常，内核通过专门的异常栈 (UXSTACK) 将现场甩回用户态处理函数。
- 在用户态申请新物理页，拷贝原始数据，并重新建立可写映射，从而实现父子进程内存的真正分离。

实验体会

本次实验主题系统调用的流程仍然十分线性，易于理解。但fork的理解难度是我认为相当大的一次。

fork本身关于父进程和子进程的诸多性质稍微有些反直觉。其代码实现尤其是写时复制机制的实现也有许多细节，在读指导书时常常有些不明所以。通读下来后理解会加深几分，但在复述中仍然会遇到不少障碍。

系统调用本身十分容易理解。但用系统调用能做到的事很多很复杂，以后还需反复阅读各种系统调用所实现功能的代码，以求更好的理解操作系统的最重要的功能。